



Assignment Title:

Investigating Process Lifecycles
and OS Interaction

Assignment Type:

Coursework : Lab Report

Module Name:

OS Security & Networks

Module Code:

ST5003CMD

Bsc (Hons) Computer Science With AI, Softwarica College of IT and E-commerce

Intake: September

Assignment Due: 26-Sept-2026

Submitted By:

CU:16537084

College ID: 250055

Name: Salman Ansari

Submitted To:

Name: Rupak Rajbanshi

Table of Content:

I. TASK 1: THE LONG-RUNNING PROCESS.....	3
A. C Source Code.....	3
B. Terminal Execution & Verification.....	3
II. TASK 2: PROCESS IDENTITY (PID AND PPID).....	4
A. C Source Code:.....	4
B. Terminal Execution & Verification:.....	6
III. TASK 3: EXIT CODES AND OS FEEDBACK.....	7
A. C Source Code.....	7
B. Terminal Execution & Verification.....	8
IV. TASK 4: STANDARD I/O STREAMS.....	9
A. C source code.....	9
B. Terminal Execution.....	9
V. TASK 5: CONDITIONAL EXECUTION AND TERMINATION.....	10
A. source code.....	10
B. Terminal Execution & Verification.....	11
VI. Conclusion.....	12

Table of Figure:

Figure 1. Program.c	4
Figure 2. Run in background	4
Figure 3. In Terminal 2 / Monitor the process	5
Figure 4. Program2 identity.c	6
Figure 5. Terminal command	7
Figure 6. Terminal command	7
Figure 7. Terminal command	8
Figure 8. Program3 input.c	9
Figure 9. Program4 input.c	10
Figure 10. Running task 4	10
Figure 11. Task 5_C-code	11
Figure 12. C-code & Compile it	12
Figure 13. Testing Control Flow	12

I. TASK 1: THE LONG-RUNNING PROCESS

A. C Source Code

- create a long-running C process that runs for approximately 30 seconds and observe the process while it is active.

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("I am starting...\n");

    // Loop for 30 seconds
    for (int i = 1; i <= 30; i++) {
        sleep(1);
    }

    printf("I am finished.\n");
    return 0;
}
```

Figure 1. Program.c

B. Terminal Execution & Verification

- Compile and run the program in the terminal to verify that it starts, runs for approximately 30 seconds, and then finishes successfully.

```
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ gcc program1.c -o program1.k
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ ls
program1.c  program1.k
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ ./program1.k
I am starting...
I am finished.
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$
```

Figure 2. Run in background

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ cd Lab_3
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ ps aux | grep task1
salman    44043  0.0  0.0 10132 2068 pts/0    S+   11:26   0:00 grep --color=auto task1
```

Figure 3. In Terminal 2 / Monitor the process

II. TASK 2: PROCESS IDENTITY (PID AND PPID)

- The PID uniquely identifies a process, while the PPID identifies the process that created it.

A. C Source Code:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    // Get current Process ID
    pid_t my_pid = getpid();

    // Get Parent Process ID
    pid_t my_ppid = getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);

    printf("Sleeping for 20 seconds...\n");
    sleep(20);

    return 0;
}
```

Figure 4. Program2 identity.c

B. Terminal Execution & Verification:

```
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ nano program2.c
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ gcc program2.c -o program2.d
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ ls
program1.c  program1.k  program2.c  program2.d
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ ./program2.d
My PID is: 44309
My Parent PID is: 35556
Sleeping for 20 seconds...
```

Figure 5. Terminal command

```
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ ./program2.d
My PID is: 44386
My Parent PID is: 43608
Sleeping for 20 seconds...
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ ps -p 12345 -o pid,ppid,cmd
  PID   PPID  CMD
  12345  12345  ps
```

Figure 6. Terminal command

III. TASK 3: EXIT CODES AND OS FEEDBACK

- A program returns an exit code to the OS, where 0 means success and a non-zero value means failure, and the shell stores it in \$?.

A. C Source Code

```
#include <stdio.h>

int main() {
    int num;

    printf("Enter a number (positive for success, negative for fail): ");
    scanf("%d", &num);

    if (num > 0) {
        printf("Success\n");
        return 0;
    } else {
        printf("Failure\n");
        return 1;
    }
}
```

Figure 7. Terminal command

B. Terminal Execution & Verification

```
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ nano program3.c
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ gcc program3.c -o program3.y
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ ./program3.y
Enter a number (positive for success, negative for fail): 5
Success
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ echo $?
0
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ ./program3.y
Enter a number (positive for success, negative for fail): -5
Failure
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ echo $?
1
```

Figure 8. Program3 input.c

IV. TASK 4: STANDARD I/O STREAMS

- `stdin` receives input from the user, while `stdout` displays program output, with `scanf()` reading from `stdin` and `printf()` writing to `stdout`.

A. C source code

```
#include <stdio.h>

int main() {
    char name[50];

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Hello, %s! Welcome to OS Class.\n", name);

    return 0;
}
```

Figure 9. Program4 input.c

B. Terminal Execution

```
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ nano program4.c
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ gcc program4.c -o program4.w
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ ./program4.w
Enter your name: salman
Hello, salman! Welcome to OS Class.
```

Figure 10. Running task 4

V. TASK 5: CONDITIONAL EXECUTION AND TERMINATION

- A program makes decisions based on input and returns an exit code to tell the OS whether it completed successfully or failed.

A. source code

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;

    printf("Current PID: %d\n", getpid());

    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0;
    } else {
        printf("Exiting...\n");
        return 1;
    }
}
```

Figure 11. Task 5_C-code

B. Terminal Execution & Verification

```
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ nano program5.c
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ gcc program5.c -o program5.z
```

Figure 12. C-code & Compile it

```
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ ./program5.z
Current PID: 45060
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ echo $?
0
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ ./program5.z
Current PID: 45067
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting...
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab_3$ echo $?
1
```

Figure 13. Testing Control Flow

VI. Conclusion

This lab gave us a practical understanding of how programs interact with the operating system. We learned that every process has a life cycle: it starts, runs, and ends. Each process has a unique PID and a parent process identified by the PPID, which helps the OS track processes. We also learned that programs use exit codes to report their status to the OS, where `0` means success and a non-zero value means an error or failure. Finally, we saw how `stdin` and `stdout` allow users to interact with programs. Overall, this lab helped us understand that the OS actively manages processes from start to finish.

